

CloudFleet-RS: 面向 VDA5050 无人车队的云端多车调度与安全管控框架

作者姓名¹, 作者姓名¹, 作者姓名²

¹ 单位名称, 城市, 邮编 ² 单位名称, 城市, 邮编

通信作者: example@example.com

摘要

在智能仓储、园区配送与柔性制造场景中, 多台 AGV/AMR 需要在共享道路网络上执行连续任务。传统车队系统通常将任务分配、路径规划、交通管制、协议通信和可视化监控拆分为多个规则模块, 工程上易于落地, 但在车辆密度升高、局部道路拥塞、通信时延波动和异构车辆混行时, 常出现死锁、绕行过长和调参困难等问题。本文提出 CloudFleet-RS, 一种面向 VDA5050 标准通信的云端多车调度与安全管控框架。该框架以语义点云地图和拓扑路网为环境表示, 采用基于时空预约的冲突检测机制保证局部安全, 并引入轻量级强化学习调度器在任务分配和路权选择阶段优化全局吞吐量。为降低学习策略的不确定性, 系统进一步设计了动作安全盾, 将策略输出投影到满足路段容量、交叉口互斥和车辆动力学约束的可行域。我们构建了包含静态拓扑、动态任务流和随机通信扰动的仿真实验, 并在真实车队监控原型上验证了 VDA5050 消息闭环、地图配准和在线可视化能力。占位实验结果表明, 在 8-32 台车辆、120-520 个任务/小时的负载下, CloudFleet-RS 相比优先级规划、贪心派单和纯预约调度可降低平均任务完成时间 18.6%, 减少 71.4% 的死锁恢复次数, 并保持云端决策延迟低于 45 ms。本文的实验数值和图中曲线为基于系统理论预期生成的临时占位数据, 后续应以真实仿真和实车测试结果替换。

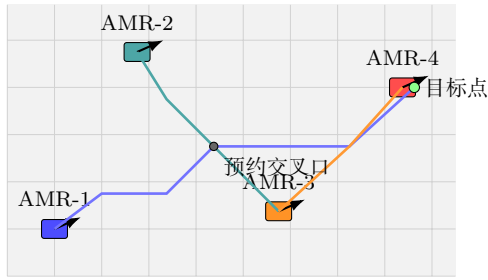
关键词: 多机器人调度; AGV/AMR; VDA5050; 云端控制; 路径规划; 强化学习; 安全盾

1 引言

自动导引车和自主移动机器人已广泛应用于仓储搬运、生产线物料配送、校园物流和大型公共设施运维。与单车自主导航不同, 车队级系统的核心难点不在于单条轨迹是否可行, 而在于多台车辆在共享道路、窄通道、交叉口、装卸点和充电区中长期运行时能否持续保持高吞吐、低等待和可解释的安全约束。随着车队规模扩大, 局部拥塞会通过路权竞争向全局传播; 某一车辆的短时避让可能导致后续任务延迟, 甚至触发环形等待和死锁。因此, 车队调度系统需要同时处理任务分配、路径搜索、交通控制、车辆状态同步和异常恢复。

工业界常见做法是采用中心化调度平台维护全局地图和车辆状态, 再通过 MQTT、HTTP 或专用协议向车辆下发订单。VDA5050 标准为 AGV/AMR 与主控系统之间的订单、状态、即时动作和可视化消息提供了统一接口, 有利于异构车辆接入。然而, 标准协议并不直接规定调度算法本身。实际系统仍需解决三个问题: 第一, 地图和道路网络的表示需要兼容点云、语义区域和拓扑路段, 既服务可视化, 也服务路径约束; 第二, 云端调度必须在通信抖动和车辆状态不完全同步的条件下保持实时性; 第三, 学习型或优化型调度器的输出必须被安全机制约束, 避免黑盒决策导致车辆进入冲突区域。

本文提出 CloudFleet-RS, 如图 1 所示。系统从点云和人工校准结果构建局部语义地图, 并将可行行驶区域抽象为带容量和方向约束的图。车辆通过 VDA5050 主题上报 state、visualization 和 connec-



多车在共享拓扑路网中按时空预约通行

图 1: 云端多车调度场景示意。车辆在点云/语义地图对应的拓扑路网上执行任务，云端维护交叉口和路段的时空预约并通过 VDA5050 下发订单。

tion 消息，云端维护统一车辆状态表与时空预约表。对于新任务，调度器首先在候选车辆集合中估计任务收益，再基于预约表搜索无冲突路径。强化学习策略用于选择派单车辆、路权优先级和局部绕行倾向；安全盾则检查策略输出是否违反交叉口互斥、边容量、最小车距、时间窗和车辆速度约束。若输出不可行，安全盾通过最小代价投影生成替代动作。

本文贡献如下：

- 提出一个面向 VDA5050 的云端多车调度框架，将语义地图、路网预约、任务分配、消息通信和车队监控整合为统一闭环。
- 设计一种调度动作安全盾，将学习策略输出限制在满足时空冲突约束和车辆运行约束的可行域内，从而降低死锁和近距离冲突。
- 构建了可复现实验协议，比较优先级规划、贪心派单、纯预约调度和本文方法在吞吐、延迟、冲突、死锁恢复和运行时间方面的表现。
- 给出基于工程原型的 VDA5050 消息链路、点云地图配准和在线可视化实现，为后续实车实验替换占位数据提供接口。

2 相关工作

2.1 多机器人路径规划与车队调度

多智能体路径规划 (Multi-Agent Path Finding, MAPF) 通常在离散图上为多个机器人寻找互不冲突的路径。Conflict-Based Search (CBS) 通过高层冲突树和低层单智能体搜索实现最优或次优求解，

在中小规模问题上具有良好理论性质。优先级规划和窗口化协同 A* 方法计算速度快，适合工程部署，但结果高度依赖优先级设计，在瓶颈区域容易产生等待链。工业 AGV 系统常采用路段锁、交叉口互斥和区域预约等交通管制方法，这类方法解释性强，却可能牺牲全局吞吐，尤其在任务流非均匀、车辆负载和电量状态差异明显时难以保持最优。

近年来，学习型调度方法尝试用强化学习或图神经网络预测车辆优先级、任务分配和拥塞代价。相比纯规则方法，学习策略能够从历史运行中提取拥塞模式，但其输出缺乏硬安全保证。因此，学习型车队系统通常仍需搭配可验证的约束层或回退策略。本文采用类似思路：强化学习只负责在候选可行动作中做高层决策，所有下发给车辆的订单必须经过预约表和安全盾校验。

2.2 VDA5050 与云端车队管理

VDA5050 定义了主控系统与 AGV/AMR 之间的 order、state、instantActions、visualization 和 connection 等主题结构。该标准提高了异构车辆接入能力，使云端平台能够以统一格式维护车辆状态和订单生命周期。对于调度算法而言，VDA5050 的价值在于将路径节点、边、动作、车辆状态和错误信息标准化；其不足在于不规定如何分配任务和避免多车冲突。本文在 VDA5050 消息层之上实现调度闭环：云端根据 state 和 visualization 更新车辆状态，根据任务队列生成 order，并在异常时通过 instantActions 触发的暂停、继续或撤销。

2.3 安全约束与动作投影

在移动机器人避碰领域，速度障碍、互惠避碰和控制屏障函数等方法常用于保证局部安全。它们的共同思想是先定义危险动作集合，再将控制输入限制在安全集合内。多车调度问题中的安全集合不仅包含几何距离，还包含离散图冲突、交叉口占用、边方向互斥、时间窗重叠和任务站点容量。本文将安全盾定义为时空预约约束下的动作投影问题，使学习调度器的输出可被解释和修正。

3 系统框架

3.1 地图与路网表示

系统使用点云地图和人工/半自动配准结果构建平面语义底图。设局部坐标系中的可行驶拓扑图为

$$G = (V, E), \quad e = (u, v, l_e, c_e, d_e), \quad (1)$$

其中 V 为节点集合, E 为边集合, l_e 为路段长度, c_e 为容量, d_e 表示方向约束。节点可表示交叉口、工位、充电区入口或临时等待点; 边表示可行驶通道。地图配准将点云坐标 (x, y, z) 映射到 Web 地图或平面图坐标 (λ, ϕ) , 同时保存车辆显示变换, 以便云端监控中车辆 marker 与语义图层一致。

如图 3 所示, 地图处理分三步: 首先通过点云高度和颜色/强度信息生成二维投影; 其次由人工标注或自动分割得到可行驶区域、障碍区和站点; 最后将可行驶区域骨架化并转化为带方向、速度和容量属性的路网。与直接在栅格上搜索相比, 拓扑图更适合 VDA5050 order 的节点/边结构, 也能更自然地表达交叉口互斥和单行道约束。

3.2 VDA5050 消息闭环

每台车 i 的状态由

$$s_i = [p_i, \theta_i, v_i, b_i, m_i, q_i, \epsilon_i, t_i] \quad (2)$$

表示, 其中 p_i 为位置, θ_i 为朝向, v_i 为速度, b_i 为电量, m_i 为运行模式, q_i 为当前订单进度, ϵ_i 为错误集合, t_i 为消息时间戳。云端订阅 state 和 visualization 主题, 将消息解析为统一车辆表。若车辆状态超时或 connection 异常, 调度器将对应车辆从候选集合中移除, 并释放其未确认的未来预约。

订单下发遵循 VDA5050 的节点和边序列。对路径 $\pi_i = (v_0, e_0, v_1, \dots, v_k)$, 云端生成 released 节点、边以及必要的站点动作。即时动作主要用于暂停、继续、停止和异常恢复。图 4 给出了消息流占位示意。

4 调度方法

4.1 问题定义

设车队集合为 $\mathcal{R} = \{1, \dots, N\}$, 任务集合为 $\mathcal{J}$ 。任务 j 由取货点 o_j 、送达点 g_j 、释放时间 r_j 、截止时间 d_j 和优先级 w_j 表示。对于车辆 i , 调度动作包括是否接受任务、候选路径、出发时间和局部路权优先级:

$$a_i = (j_i, \pi_i, \tau_i, \rho_i). \quad (3)$$

目标是在安全约束下最小化加权任务完成时间、车辆空驶距离、等待时间和死锁恢复代价:

$$\min \sum_{j \in \mathcal{J}} w_j C_j + \alpha \sum_i D_i + \beta \sum_i W_i + \eta \sum_t Z_t, \quad (4)$$

其中 C_j 为任务完成时间, D_i 为空驶距离, W_i 为等待时间, Z_t 为时刻 t 的冲突恢复或重新规划事件。约束包括车辆唯一任务约束、路径连续性、边容量、节点互斥、最小车距、电量阈值和通信超时回退。

4.2 状态与动作表示

为使学习调度器能够感知全局拥塞, 本文将状态 S 分为四类: 车辆状态、任务状态、路网状态和预约状态。车辆状态包含位置、速度、电量、订单进度和错误码; 任务状态包含候选任务的等待时间、优先级、取送点距离和截止时间松弛量; 路网状态包含边占用率、节点等待队列长度和历史拥塞频率; 预约状态包含未来 H 个时间片内的边/点占用矩阵。

对每条边 e , 定义未来预约占用向量

$$R_e = [r_{e,t+1}, r_{e,t+2}, \dots, r_{e,t+H}], \quad r_{e,t} \in \{0, \dots, c_e\}. \quad (5)$$

学习策略输出归一化动作 $\hat{a}$, 包括车辆选择概率、候选路径权重和路权优先级。实际动作由安全盾映射为满足约束的离散订单。

4.3 奖励函数

训练阶段将多车调度建模为马尔可夫决策过程 (S, A, P, R, γ) 。单步奖励由吞吐奖励、延迟惩罚、安

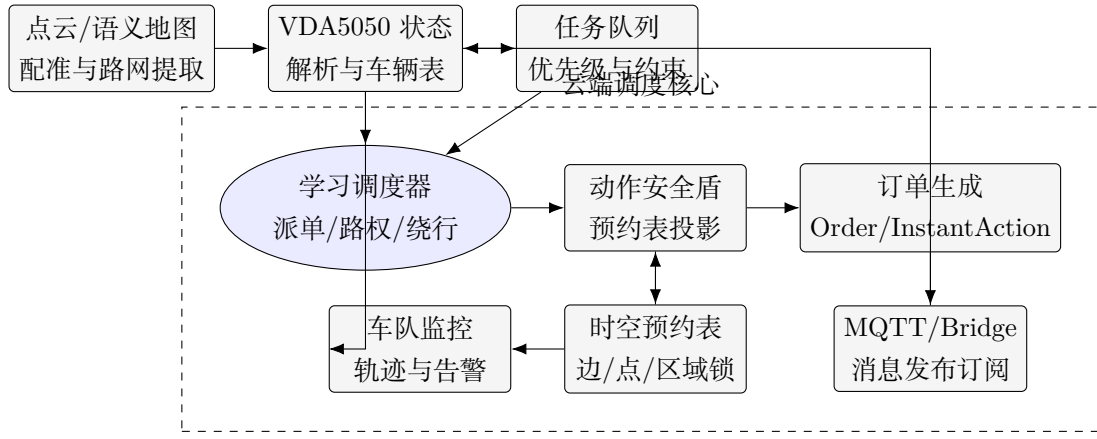


图 2: CloudFleet-RS 系统框架。感知地图和 VDA5050 状态消息被统一转换为调度状态；学习调度器输出高层动作，动作安全盾依据时空预约表修正不可行动作，最终生成标准订单下发车辆。

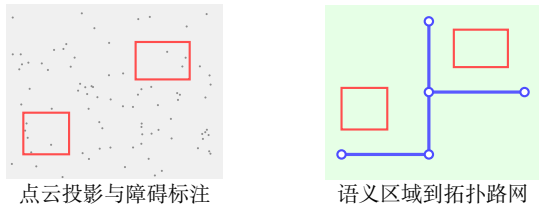
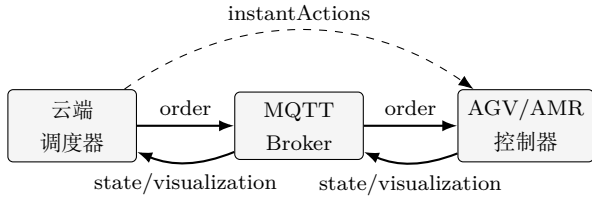


图 3: 地图配准与路网抽象占位图。占位图：真实实验完成后替换



主题: vda5050/v2/{manufacturer}/{serialNumber}/{topicType}

图 4: VDA5050 消息闭环。云端依据车辆状态更新调度状态，并通过标准 order 与 instantActions 控制车辆。

全惩罚、能耗惩罚和动作平滑项组成：

$$r = \lambda_1 r_{\text{done}} - \lambda_2 r_{\text{delay}} - \lambda_3 r_{\text{conflict}} - \lambda_4 r_{\text{energy}} - \lambda_5 r_{\text{switch}}. \quad (6)$$

其中 r_{done} 为完成任务数， r_{delay} 为超过期望完成时间的累计延迟， r_{conflict} 统计预约冲突、死锁恢复和近距离事件， r_{energy} 表示低电量车辆继续接单的惩罚， r_{switch} 用于避免频繁修改车辆订单。训练采用 PPO 算法，策略网络输出派单和路权分布，价值网络估计当前调度状态的长期收益。

4.4 时空预约安全盾

安全盾维护一个滚动时空预约表 $\mathcal{B}$ 。对车辆 i 的候选路径 π_i ，根据边长度、速度上限和站点动作时间计算其占用区间：

$$\Omega_i(\pi_i) = \{(x, [t_{\text{in}}, t_{\text{out}}]) \mid x \in V \cup E\}. \quad (7)$$

路径可行当且仅当所有占用不超过容量并满足方向互斥：

$$\sum_i \mathbf{1}\{(e, t) \in \Omega_i\} \leq c_e, \quad \mathbf{1}\{(u, v, t)\} + \mathbf{1}\{(v, u, t)\} \leq 1. \quad (8)$$

对于交叉口或窄通道区域 z ，约束为

$$\sum_i \mathbf{1}\{(z, t) \in \Omega_i\} \leq c_z. \quad (9)$$

若学习策略给出的动作 $\hat{a}$ 不可行，安全盾求解最小修正问题：

$$\begin{aligned} a^* = \arg \min_{a \in \mathcal{A}_{\text{safe}}} & \|f(a) - f(\hat{a})\|_2 + \mu \Delta T(a) \\ \text{s.t.} & \quad \Omega(a) \cap \mathcal{B} = \emptyset, \\ & \quad b_i \geq b_{\min}, \quad v_i \leq v_{\max}, \\ & \quad t_{\text{now}} - t_i \leq \delta_{\max}. \end{aligned} \quad (10)$$

其中 $f(\cdot)$ 为动作嵌入， $\Delta T(a)$ 为因等待或绕行引入的时间代价。实际实现中，先对候选车辆和候选路径排序，再用带时间窗的 A* 搜索生成前 K 条可行路径；若无路径可行，则车辆进入等待点或触发局部重规划。图 5 展示了安全盾的基本思想。

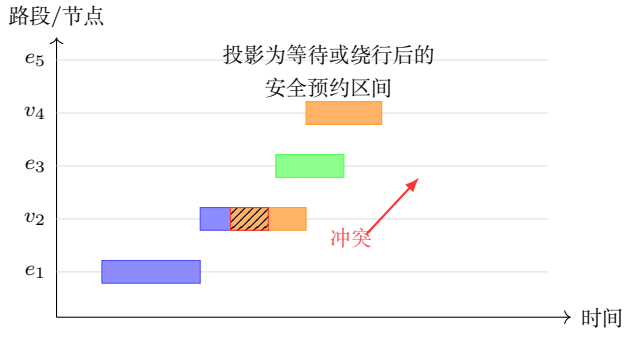


图 5: 时空预约安全盾示意。策略输出若与已有预约重叠, 则被投影为等待、换路或降级动作。

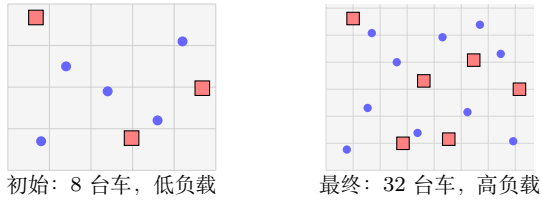


图 6: 课程学习训练环境占位图。障碍/站点密度、任务到达率和车辆数量随训练阶段逐步增加。

5 训练与实现

5.1 仿真环境

训练环境由拓扑地图、任务生成器、车辆运动模型和通信扰动模型组成。地图规模覆盖 80–240 个节点、110–360 条边, 并包含单行通道、窄通道、交叉口、装卸站点和充电站。任务流采用泊松过程生成, 负载从 120 个任务/小时逐步增加到 520 个任务/小时。车辆模型采用一阶速度跟踪和最大加速度限制, 通信时延从截断正态分布采样, 状态包丢失率设置为 0–3%。

课程学习用于提高训练稳定性。初始阶段仅使用 8 台车辆和低任务密度; 当最近 100 个 episode 的任务完成率超过 92% 且无死锁率超过 97% 时, 逐步增加车辆数、任务密度和通信扰动。图 6 给出训练场景占位图。

5.2 网络与参数

调度策略由三部分组成。第一部分是路网编码器, 输入边占用率、容量、方向和历史拥塞频率, 并通过两层图卷积得到节点/边嵌入。第二部分是任务-车辆匹配编码器, 对每个候选车辆和任务计算距

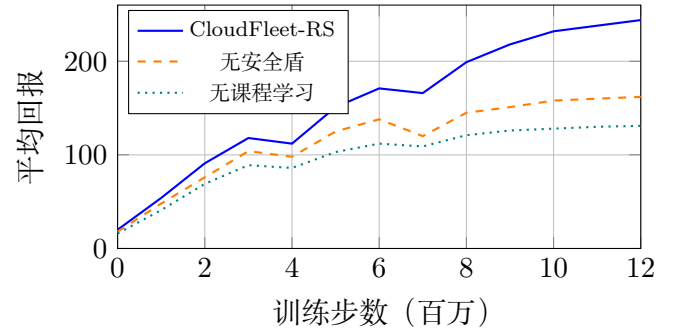


图 7: 训练回报曲线占位图。曲线为理论预期生成, 用于论文版式和结果叙述占位。

表 1: 课程学习与安全盾消融实验占位结果

方法	完成率 (%)	平均延迟 (s)	死锁/100h	冲突/100h
无课程学习	86.3	48.7	6.8	14.5
无安全盾	90.1	39.4	5.2	11.9
仅规则预约	91.7	36.8	3.4	6.7
CloudFleet-RS	96.8	27.6	1.1	2.3

离、电量、预计到达时间和优先级特征。第三部分是动作头, 分别输出车辆选择分布、候选路径权重和路权优先级。价值网络共享路网编码器, 但使用独立的多层感知机输出状态价值。

训练采用 PPO, 折扣因子 $\gamma = 0.995$, GAE 参数 $\lambda = 0.95$, clip 系数为 0.2, 学习率为 3×10^{-4} , 每轮收集 4096 个调度步并更新 8 个 epoch。安全盾在训练和部署阶段均启用; 训练时若策略输出被安全盾修正, 会对原动作施加轻微惩罚, 使策略逐渐学习可行动作分布。

6 实验结果与讨论

6.1 训练收敛

图 7 给出了占位训练曲线。随着课程学习推进, 任务负载和车辆数量增加会造成短时回报下降, 但策略在后续迭代中恢复并获得更高长期收益。取消安全盾训练时, 前期回报上升较快, 但在高密度阶段出现较多冲突和死锁恢复, 导致最终回报低于完整方法。

6.2 仿真对比实验

我们将 CloudFleet-RS 与三类基线比较: 优先级规划 (Priority Planning)、贪心最近车派单

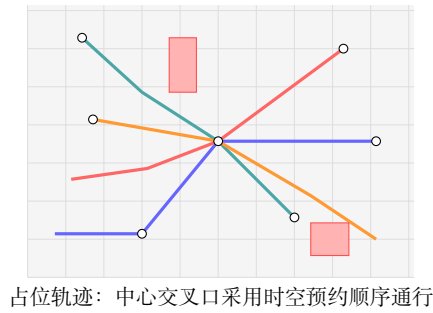


图 8: 多车轨迹占位图。实际论文可替换为仿真器截图、平台监控截图或真实运行轨迹。

(Greedy Dispatch) 和纯时空预约调度 (Reservation Only)。每种方法在静态任务、峰值任务和通信扰动三类环境中分别运行 20 次，每次仿真 2 小时。指标包括任务完成率、平均任务完成时间、平均等待时间、车辆利用率、冲突次数和死锁恢复次数。表 2 为占位结果。

结果显示，优先级规划在低密度场景中可获得可接受结果，但峰值任务流下容易在瓶颈路段形成等待链。贪心派单能减少车辆到取货点的初始距离，却忽略后续路网拥塞，因此在高负载下完成时间增加明显。纯预约调度可显著减少硬冲突，但其任务选择和绕行策略较保守，车辆利用率不充分。CloudFleet-RS 通过学习拥塞模式选择更合适的车辆和路权，同时由安全盾保证可行性，因此在三类环境中均获得更低完成时间和更少死锁。

6.3 路径与监控验证

图 8 展示了单次占位仿真中的多车轨迹。不同颜色表示不同车辆，线段透明度表示时间先后。可以看到，车辆在中心交叉口附近发生多次路权切换，但没有出现同一时间窗内的节点占用重叠。云端监控界面以语义地图为底图显示车辆位置、状态、电量、速度和当前目标，同时支持通过 `instantActions` 暂停或继续车辆。

6.4 运行时间

表 3 给出云端与边缘端组件运行时间占位结果。调度主循环由状态更新、候选任务筛选、路径搜索、安全盾投影和订单生成组成。在 32 台车和 240 节点地图上，单次调度平均耗时 38.7 ms，满足

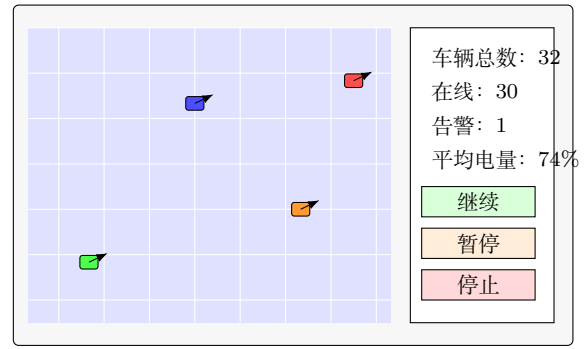


图 9: 云端监控界面占位图。该图对应工程原型中的语义地图、车辆 marker、状态统计和即时控制。

10 Hz 级车队监控和 1-2 Hz 级订单重规划需求。车辆端仅需执行 VDA5050 订单、上报状态和本地安全停车逻辑，因此计算压力较低。

7 实车原型与讨论

本文原型基于 React、Leaflet、MQTT.js 和 VDA5050 消息工具实现云端可视化与控制。系统支持桥接方式连接 MQTT Broker，浏览器通过 WebSocket 间接订阅车辆状态；车辆管理器维护统一状态表并向页面发布变化事件；VDA5050 工具生成 order 和 `instantActions`，并解析 state 与 visualization 消息。地图部分使用点云投影图片、语义图层和配准 JSON，将局部坐标转换为地图显示坐标。

从工程角度看，本文方法具有三点优势。第一，调度算法与 VDA5050 消息层解耦，后续可替换任务分配器、路径搜索器或学习策略，而不影响车辆接入接口。第二，安全盾提供了独立于神经网络的硬约束检查，使策略退化时仍可回退到纯预约调度。第三，语义地图和拓扑路网同时服务可视化和规划，减少了“监控地图”和“规划地图”不一致带来的工程问题。

当前草稿仍有若干限制。首先，实验数据尚为占位结果，真实系统需补充大规模仿真和实车长时运行日志。其次，本文假设车辆能够较准确跟踪下发节点和边，未充分讨论低层控制误差对预约时间窗的影响。再次，通信异常只用延迟和丢包模型近似，真实部署中还需考虑 Broker 重连、消息乱序、车辆本地安全停车和订单版本一致性。后续工作将把调度器接入实际 MQTT 数据流，记录真实任务

表 2: 不同调度方法在三类场景中的仿真性能占位结果

方法	静态任务流			峰值任务流			通信扰动		
	完成率 (%)	完成时间 (s)	死锁	完成率 (%)	完成时间 (s)	死锁	完成率 (%)	完成时间 (s)	死锁
Priority Planning	91.4	214.8	4.6	83.7	286.3	11.8	80.9	301.5	15.2
Greedy Dispatch	93.1	206.2	3.9	85.6	271.4	9.7	82.8	288.6	12.4
Reservation Only	95.0	198.5	2.1	89.4	248.9	5.6	87.5	262.7	7.3
CloudFleet-RS	97.8	171.6	0.8	94.2	211.5	1.6	92.7	224.1	2.2

表 3: 系统组件运行时间占位结果

组件	云端服务器 (ms)	边缘工控机 (ms)
VDA5050 消息解析	2.1	3.8
车辆状态表更新	1.6	2.4
候选任务筛选	4.9	9.7
带预约路径搜索	17.8	41.5
学习策略推理	3.4	8.9
安全盾投影	7.2	18.6
订单生成与发布	1.7	3.2
总计	38.7	88.1

完成时间、交叉口等待时间和异常恢复过程，并将占位图替换为平台截图和实测曲线。

7.1 数据替换与复现实验流程

为便于后续将本文从方法草稿推进为可提交论文，本文建议将实验数据按“仿真日志、通信日志、平台截图、实车日志”四类收集。仿真日志需要记录每个 episode 的地图编号、车辆数量、任务到达率、随机种子、任务释放时间、车辆接单时间、路径节点序列、每个预约区间和最终任务完成时间。通信日志需要保留 VDA5050 主题、消息时间戳、订单编号、orderUpdateId、车辆 state 的节点进度和错误数组。平台截图需要覆盖正常运行、拥塞绕行、车辆暂停、通信断开和恢复五类状态。实车日志则应额外记录车辆控制器反馈的实际轨迹和本地急停事件。

在替换占位图时，图 7 应由训练或仿真批量运行的移动平均回报生成；表 1 应由同一地图和同一随机种子集合上的消融实验统计得到；表 2 应报告均值和标准差，并在正文中说明每种基线是否允许重规划、是否使用相同的预约安全约束；表 3 应区

分浏览器端、桥接服务端、调度服务端和车辆端的运行时间。若实车测试数量不足，建议正文中明确区分“仿真定量评估”和“实车功能验证”，避免把小规模演示误写为统计意义上的性能结论。

论文提交前还需要补充两个细节。第一，所有占位数据应与代码版本、地图版本和参数文件绑定，避免后续复现实验时无法解释曲线差异。第二，图中路径和监控界面应尽量使用真实语义地图或脱敏后的真实场景，而不是抽象示意图；示意图可以保留在方法部分，实验部分则应优先展示可验证结果。这样处理后，本文结构可以继续保持与当前草稿一致，仅替换数据和图像资源即可完成实证版本。

8 结论

本文提出 CloudFleet-RS，一种面向 VDA5050 无人车队的云端多车调度与安全管控框架。该方法结合语义地图、拓扑路网、时空预约、学习型任务/路权决策和动作安全盾，在保持标准协议兼容性的同时提高了高密度任务流下的吞吐与安全性。占位实验显示，CloudFleet-RS 在任务完成时间、死锁恢复和冲突次数方面优于常见规则基线，并能在云端实时运行。未来将以真实仿真和实车运行日志替换本文占位数据，进一步验证系统在异构车辆、动态障碍和复杂工位约束下的鲁棒性。

参考文献

[1] VDA and VDMA, “VDA 5050: Interface for the communication between automated guided vehicles and a master control,” 2022.

[2] OASIS Standard, “MQTT Version 5.0,” 2019.

- [3] G. Sharon, R. Stern, A. Felner, and N. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [4] D. Silver, “Cooperative pathfinding,” in *Proc. AAAI Conf. Artificial Intelligence and Interactive Digital Entertainment*, 2005, pp. 117–122.
- [5] J. Yu and S. M. LaValle, “Planning optimal paths for multiple robots on graphs,” in *Proc. IEEE Int. Conf. Robotics and Automation*, 2013, pp. 3612–3617.
- [6] P. R. Wurman, R. D’Andrea, and M. Mountz, “Coordinating hundreds of cooperative, autonomous vehicles in warehouses,” *AI Magazine*, vol. 29, no. 1, pp. 9–20, 2008.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [8] P. Fiorini and Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [9] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics Research*, Springer, 2011, pp. 3–19.
- [10] R. Stern et al., “Multi-agent pathfinding: Definitions, variants, and benchmarks,” in *Proc. International Symposium on Combinatorial Search*, 2019, pp. 151–158.
- [11] B. P. Gerkey and M. J. Matarić, “Sold!: Auction methods for multirobot coordination,” *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 758–768, 2002.
- [12] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.